



Security Audit

Avail (Infrastructure Layer)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Avail partnered with Hashlock to conduct a security audit of their L1 and L2 validator smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Avail is a Web3 infrastructure layer designed to enable modular execution layers to scale and interoperate in a trust-minimized manner. Its Data Availability (DA) Mainnet is now live, marking the first step toward Web3 unification. Avail's roadmap includes Nexus, a permissionless verification hub to unify rollups and chains, and Fusion Security, which aims to provide strong, shared economic security. The project offers resources such as technical documentation, guides, and a testnet explorer for developers interested in building on its platform.

Project Name: Avail

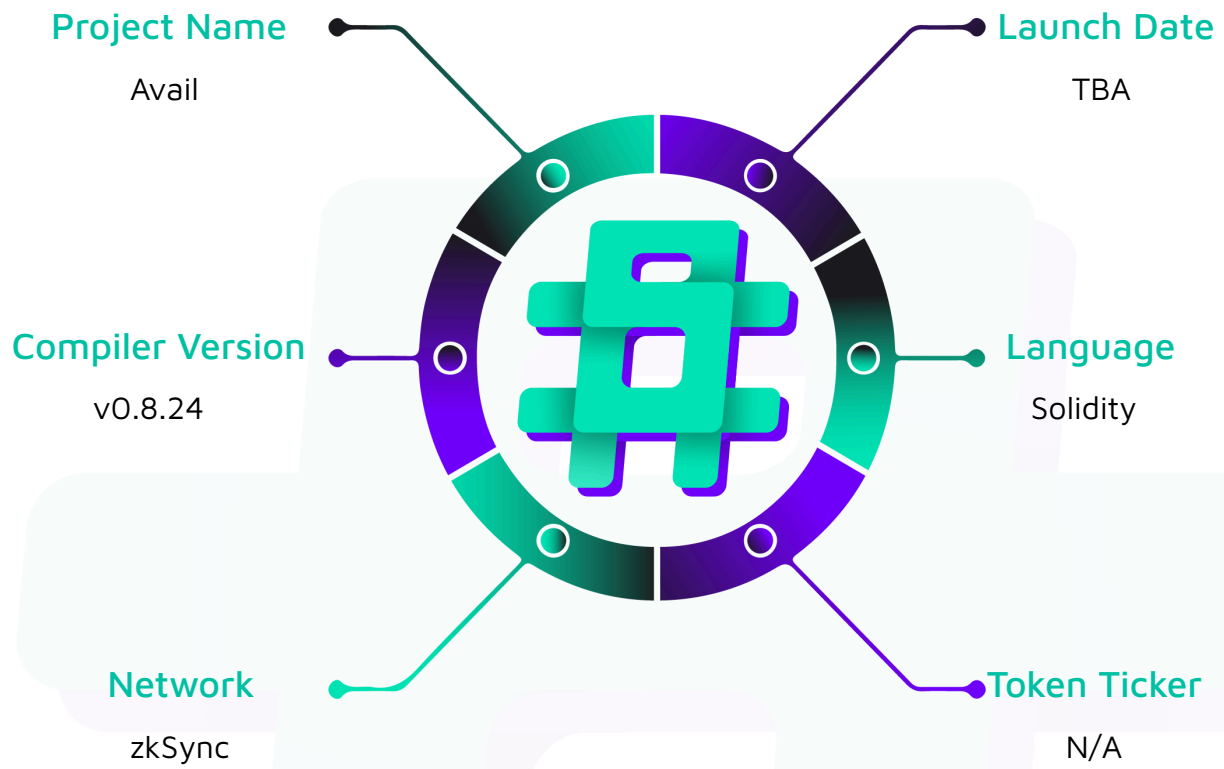
Project Type: ZK Cross Chain Communication

Compiler Version: 0.8.24

Website: <https://www.availproject.org/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Avail Project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Avail Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	29 January, 2025
Contract 1	AvailAttestationLib.sol
Contract 1 MD5 Hash	7626e16672d22791dd123d48e050e112
Contract 2	AvailL1DAValidator.sol
Contract 2 MD5 Hash	c91a509e3d03965b6101e2c6c3197b96
Contract 3	AvailL2DAValidator.sol:
Contract 3 MD5 Hash	fb409e72660c0f1d98e22199f93ea05c
Contract 4	DummyAvailBridge.sol
Contract 4 MD5 Hash	3e9cfdad10d4d5b9c66203eb53acc4b8
Contract 5	DummyVectorX.sol
Contract 5 MD5 Hash	e919e6bca94a548e0e4c681675a69582
Contract 6	IAvailBridge.sol
Contract 6 MD5 Hash	598a3c64f42666c50a8653a27deceb59
Contract 7	IVectorx.sol
Contract 7 MD5 Hash	f95fd87dad940e2b89c6efe8bf95d9c0
GitHub Commit Hash	68ddadf8106bde020368e6ab5613a522a88f58f6

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the contracts are completely in the purview of existing zkSync bridge contracts. We initially identified some vulnerabilities that have since been acknowledged.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AvailAttestationLib.sol - This contract serves as a base library for verifying and storing attestations from the Avail data availability layer. It should: - Store attestation data for state recovery - - Interface with the Avail bridge to verify proofs - Track block numbers and leaf indices for attestations - Provide functionality for verifying inclusion proofs	Contract achieves this functionality.
AvailL1DAValidator.sol - Verify inclusion proofs during commitBatches - Check that L2 commitments match inclusion proofs - Call DA attestation bridge for verification - Prevent batch commitment if verification fails - Process operator DA input and validate against L2 output hash	Contract achieves this functionality.
AvailL2DAValidator.sol - Generate commitments for batch data (pubdata) - Process L2->L1 logs, messages, and bytetimes - Create verifiable commitments for the DA layer - Handle state diffs and total pubdata	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Avail project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Avail project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] AvailL1DAValidator - Unrestricted Access to Data Availability Validation Function

Description

The `checkDA` function in `AvailL1DAValidator` lacks access control mechanisms, allowing any external address to call this critical function. This function is specifically designed to be called only by the Executor contract as confirmed by the development team. The absence of access control creates serious risks to the Validium system's security and data integrity.

When this function is called, it:

- Validates L2 commitments
- Verifies data availability proofs
- Adds permanent attestations to the system's state storage

Vulnerability Details

The contract was implemented without the necessary access control modifier that would restrict function execution to the authorized Executor contract. This oversight allows unrestricted access to a critical system function that should only be accessible to trusted system components.

As long as the input parameters are valid and satisfy the verification conditions, the function will execute successfully regardless of the caller's identity. This allows unauthorized parties to manipulate the system's state by adding attestations that should only come from the legitimate Executor contract.

```
// Current implementation

function checkDA(

    uint256,

    uint256,

    bytes32 l2DAValidatorOutputHash,

    bytes calldata operatorDAInput,

    uint256 maxBlobsSupported

) external returns (L1DAValidatorOutput memory output)

// Can be called by any address
```

Impact

Without access restrictions, malicious actors could interfere with the system's normal operation by adding unauthorized attestations, potentially disrupting the chain reconstruction process, which relies on these attestations being properly ordered and validated.

Recommendation

Implement proper access control by adding an authorized Executor address and a modifier to restrict access

Note

Avail's team informed Hashlock that this finding relates to a shared contract architecture used by all rollup proposers. While it's technically feasible to restrict the checkDA function with access control (limiting calls to an authorized Executor contract), doing so would prevent other legitimate proposers from interacting with the contract as intended. Instead, the system is designed to remain open to all proposers and relies on filtering attestations based on the sender address to ensure that only valid attestations from authorized sources are processed. This design choice strikes a balance between maintaining broad accessibility for multiple rollup proposers and safeguarding the integrity of the system's state.

Status

Acknowledged



Low

[L-01] AvailL1DAValidator - Missing Input Length Validation

Description

The `checkDA` function in `AvailL1DAValidator` performs operations on `operatorDAInput` without first validating its length. The function immediately attempts to slice the first 32 bytes and then decode the remainder into a `MerkleProofInput` structure.

Vulnerability Details

If `operatorDAInput` is less than 32 bytes, the slice operation will trigger panic and revert the transaction

```
function checkDA(
    uint256,
    uint256,
    bytes32 l2DAValidatorOutputHash,
    bytes calldata operatorDAInput,
    uint256 maxBlobsSupported
) external returns (L1DAValidatorOutput memory output) {
    // Immediately tries to slice without length check
    output.stateDiffHash = bytes32(operatorDAInput[:32]);

    // Attempts to decode remaining bytes without ensuring sufficient length
    IAvailBridge.MerkleProofInput memory input = abi.decode(
        operatorDAInput[32:],
        (IAvailBridge.MerkleProofInput)
    );

    // ...
}
```



```
}
```

Impact

If `operatorDAInput` is exactly 32 bytes or has insufficient data after the first 32 bytes, the `abi.decode` operation will revert and these reverts will happen with low-level panic errors rather than clear, custom error messages

Recommendation

Add explicit length validation before processing the input:

```
function checkDA(
uint256,
uint256,
bytes32 l2DAValidatorOutputHash,
bytes calldata operatorDAInput,
uint256 maxBlobsSupported
) external returns (L1DAValidatorOutput memory output) {
    // Custom error for invalid input length
    error InvalidInputLength();

    // Ensure minimum length for state diff hash (32 bytes)
    if (operatorDAInput.length < 32) {
        revert InvalidInputLength();
    }

    // ... Remaining code
```

Status

Acknowledged

[L-02] AvailAttestationLib - Implicit Assumptions in Type Casting

Description

The `AvailAttestationLib` contract performs unchecked type conversions from larger to smaller integer types when storing attestation data. While Solidity 0.8.x provides overflow protection, these implicit conversions could lead to silent data truncation, potentially causing the system to store incorrect block numbers and leaf indices. This could affect the ability to accurately reconstruct historical chain state, which is a core requirement of the system

Vulnerability Details

The vulnerability exists in the `_attest` function of `AvailAttestationLib`:

```
function _attest(IAvailBridge.MerkleProofInput memory input) internal virtual {
    if (!bridge.verifyBlobLeaf(input)) revert InvalidAttestationProof();

    attestations[input.leaf] = AttestationData(
        vectorx.rangeStartBlocks(input.rangeHash) + uint32(input.dataRootIndex) + 1,
        uint128(input.leafIndex)
    );
}
```

Consider this scenario:

```
// Given input.dataRootIndex = 2^32 + 5
uint32 truncatedValue = uint32(input.dataRootIndex); // Silently truncates to 5
uint32 blockNumber = truncatedValue + 1;              // Becomes 6 instead of the
intended value

// The attestation is stored with an incorrect block number
attestations[input.leaf] = AttestationData(
    6, // Incorrect block number due to truncation
    uint128(input.leafIndex)
```

```
);
```

Recommendation

We recommend adding explicit bounds validation before performing type conversions in the `_attest` function. The contract should check that `input.dataRootIndex` doesn't exceed `type(uint32).max - 1` and that `input.leafIndex` fits within `uint128` before performing the conversions. This can be implemented by adding simple bounds checks with custom error messages at the start of the function.

Status

Acknowledged

[L-03] Multiple - Lack of State Accounting and Validation in L2-L1 Data Flow

Description

The current implementation of `AvailL1DAValidator` doesn't maintain any accounting of processed L2 validator output hashes or validate that they're being processed in the correct order. While the system relies on the `zkSync` bridge contracts for L2->L1 message passing (as confirmed by the development team), the lack of internal state tracking makes it harder to verify the proper sequencing of data availability attestations and could complicate future system upgrades or auditing.

Vulnerability Details

The contract validates each proof in isolation without maintaining any record of previously processed L2 validator output hashes or ensuring they're processed in the intended sequence. While this might be handled by the `zkSync` bridge contracts, having internal validation would provide an additional layer of security and make the system more robust.

Recommendation

Consider adding basic state tracking and sequence validation. For example, track the processed batches.

Status

Acknowledged

QA

[Q-01] L1DAValidatorOutput - Empty Returns

Description

The `L1DAValidatorOutput` struct includes arrays that are initialized but never populated. While this is intentional for interface compatibility, it should be clearly documented to prevent confusion for developers integrating with the system.

Recommendation

Add explicit documentation about the unused fields.

Status

Acknowledged

[Q-02] Multiple contracts - Events not emitted on key state changes

Description

The contracts lack event emissions for critical state changes, particularly in the `AvailAttestationLib` and `AvailL1DAValidator` contracts. This absence of events makes it difficult to track and monitor important system operations off-chain, complicates system monitoring, and makes it harder for external systems to track the state of attestations. Events are particularly important in this system since it's designed for chain reconstruction and historical data verification. While this might be a design choice, emitting events for significant state changes is considered best practice as it aids in contract monitoring, off-chain tracking, and integration testing.

Recommendation

Consider emitting events for all functions that change key contract states to improve contract monitoring and transparency.

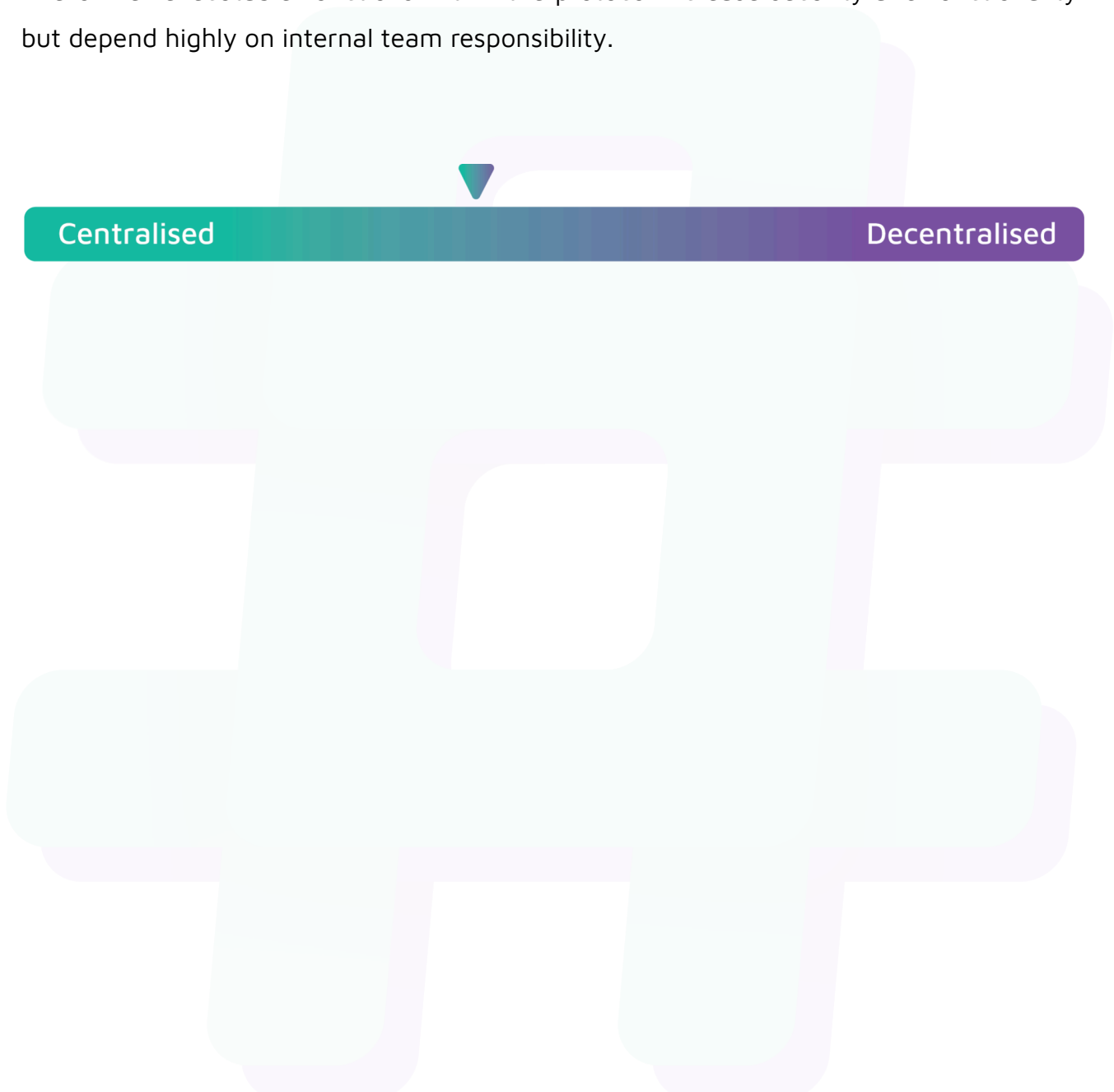
Status

Acknowledged

Centralisation

The Avail project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Avail project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.